

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

"The expert at anything was once a beginner." -Hayes

Getting Started with JDO

*T*his chapter explains the basics of how JDO works and how to use JDO to create, read, update and delete persistent objects. It introduces some of the advanced concepts behind JDO and aims to provide sufficient understanding to allow a Java programmer to develop an application using JDO. The examples for this chapter are located in directory “examples/chapterX” on the CD accompanying the book.

3.1 HOW DOES JDO WORK?

The goal of JDO is to be able to transparently store instances of any user defined Java class in a datastore and retrieve them again with as few a limitations as possible. The object instances that JDO stores and retrieves are referred to as “persistent objects”. From the application perspective these “persistent objects” appear as normal, in-memory Java objects. However underneath the covers JDO is actually making the fields of these instances persistent in a datastore – all without any explicit action on behalf of the application.

Fields, Methods and Remote Invocation

JDO has nothing to do with where methods get executed, it does not provide a means of remote method invocation ala RMI and EJB or store and execute methods in some datastore.

JDO simply specifies how the fields of a persistent object should be managed in-memory, being transparently stored and retrieved to and from an underlying datastore. With JDO, methods are invoked on an persistent object by an application within a JVM, as per any regular in-memory Java object.

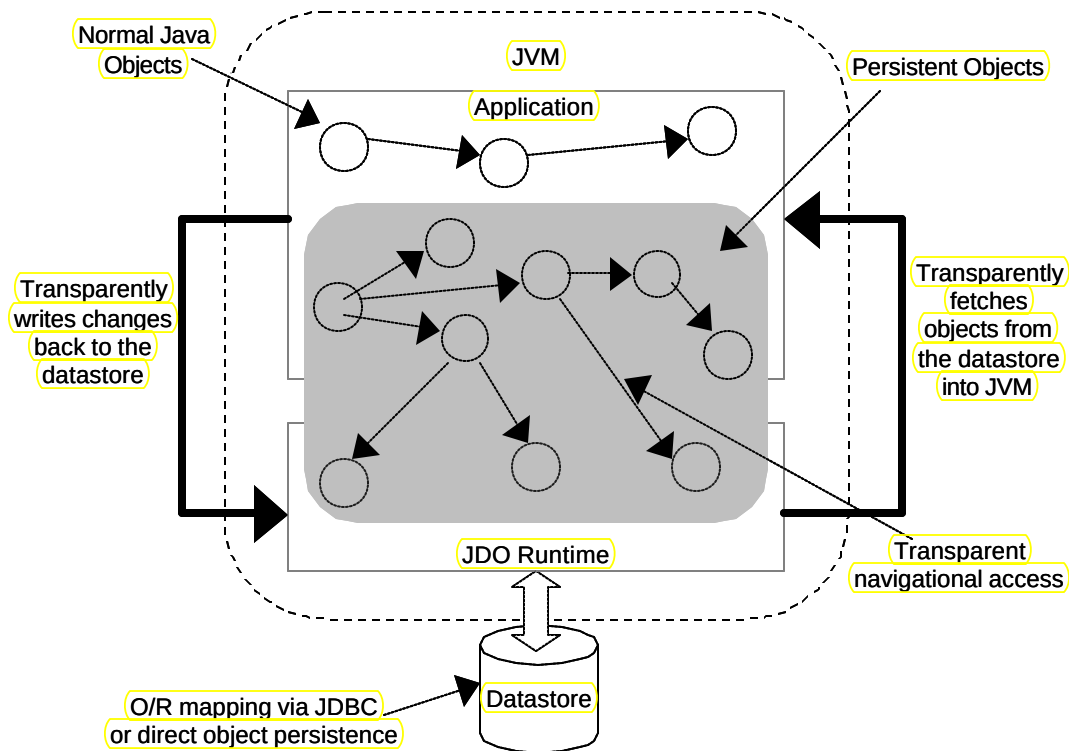


Figure 3.1: JDO Runtime Environment

Figure 3.1 provides a simple schematic of how JDO works. The JDO runtime and the application run together in the same JVM. The application delegates to the JDO runtime to retrieve the fields of persistent objects as needed. The JDO runtime tracks modifications to the fields and writes these changes back to the datastore at the end of the transaction. The JDO runtime is responsible for mapping the fields of the persistent objects to and from memory and the underlying datastore.

JDO achieves its transparency of access by defining a contract that a class has to adhere to. Any class that implements this contract can then be used with any JDO implementation.

Binary Compatibility

JDO mandates that vendors ensure binary compatibility between different JDO implementations. What this means is that a class that adheres to the JDO contract can be used with any vendor's JDO implementation, without recompilation.

It's similar to using JDBC, where a JDBC application can be run as-is using JDBC drivers from different vendors and even different relational databases. In fact it's somewhat better than this, since with JDBC an application is still prone to differences in SQL support across different databases. With JDO, SQL is not directly exposed. Even though a JDO runtime might itself use JDBC to access a relational database as its datastore, it is responsibility of the JDO vendor to resolve the differences in SQL support across databases.

Even better, unlike SQL, a JDO application can work as-is across different types of databases, not just relational: object databases; flat-files; and so on. All that is required is a vendor to provide a JDO compliant runtime.

The JDO specification defines this contract as a Java interface, called `PersistenceCapable`, and a programming style that the class implementation must follow. A class that adheres to this contract is referred to as being "persistence capable".

Persistence Capable and Persistence

A class is said to be persistence capable if its instances can be stored in a datastore by a JDO implementation. However, just because a class is persistence capable it does not mean that all its instances have to be persistent, it just means the option is there.

Whether a particular instance is persistent or not depends on the application. It's a bit like Java serialization, just because a class implements the `Serializable` interface it doesn't mean that all instances will or have to be serialized.

However, the intention of JDO is not to expect the developer to have to worry about making a class persistence capable, it's a tedious job better left to tooling.

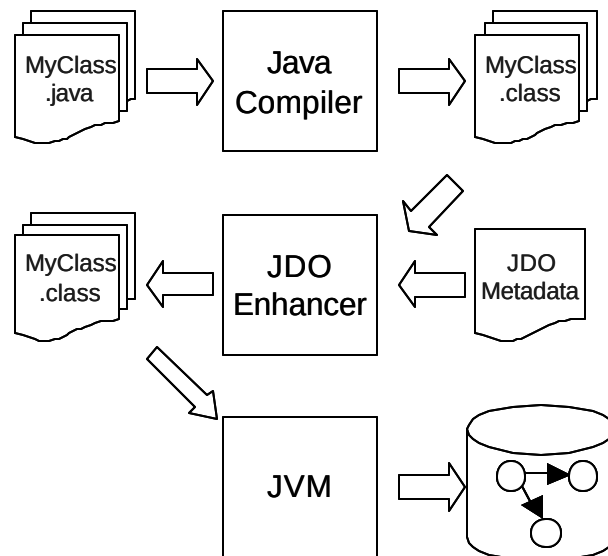
There are three main ways to create a persistence capable class:

1. **Source code generation** This is where the source code for a class is generated from scratch. This approach works well if the object model is defined in a modeling tool and is being automatically generated or the datastore schema already exists and the object model is to be generated from it. Vendor supplied tools would be used to generate source code adhering to the persistence capable contract, which would then be compiled as normal. The drawback of this approach is that it won't work for existing classes and won't appeal to those who like to write their own code.
2. **Source code pre-processing** This is where existing source code is pre-processed and updated. This approach works well if the source code for a class is available.

Vendor supplied tools would be used to read the original source code and update it to adhere to the persistence capable contract which would then be compiled as normal. The drawback of this approach is that it won't work unless the original source code is available, but it does have the benefit that a developer can write his or her own source code. Typically the pre-processing is a pre-compilation step in the build process and the generated code may be kept to aid in debugging.

3. **Byte code enhancement** This is where the compiled Java byte code for a class is enhanced directly. This approach works well even if the source code is not available. Vendor supplied tools would be used to read a class file and insert additional byte code directly to adhere to the persistence capable contract. This approach has the benefit of being completely transparent to the developer and the enhancement is simply a post-compilation step in the build process. Although JDO mandates that an enhanced class should still function correctly when debugged against the original source code some developers may be distrustful when they can't see the code for what has been changed (although they could of course always de-compile the enhanced class file afterwards).

Byte code enhancement is likely to be the most popular approach for vendors. Which means all that a developer need do is implement and compile a Java class as normal and leave it to the vendor's byte code enhancement tools to take care of the rest. The following figure



provides a schematic of how a byte code enhancement process would work:

Figure 3.2 Byte code Enhancement Process

A Java class is compiled using a Java compiler to generate a class file. The vendor's byte code enhancement tool reads the class file along with the JDO metadata for the class (this metadata is explained in [Section X.X](#)) and either updates the existing class file or creates a new one. The “enhanced” class file is then loaded by a JVM along with the JDO runtime and the application. The application can then transparently store instances of the persistence capable class in the datastore.

3.2 THE JDO BASICS

The JDO specification defines twenty or so classes and interfaces in total, but really there are just five main classes that a developer needs to know (which are actually Java interfaces) and the class diagram in Figure 3.3 shows how the interfaces are related:

- PersistenceManagerFactory
- PersistenceManager
- Extent
- Query
- Transaction

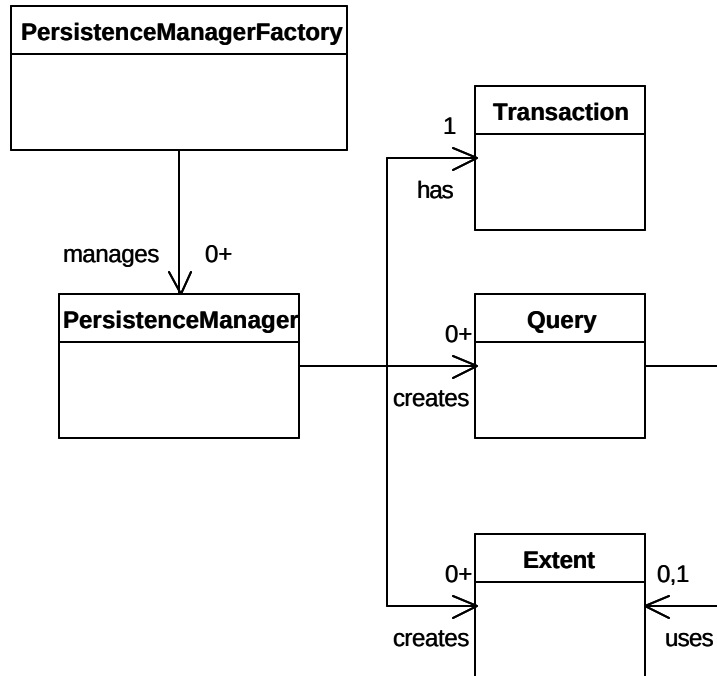


Figure 3.3 JDO Class diagram

A **PersistenceManagerFactory** is used to get a **PersistenceManager** instance. There is a one-to-many relationship between **PersistenceManagerFactory** and **PersistenceManager**. A **PersistenceManagerFactory** can create and manage many **PersistenceManager** instances and even implement pooling of **PersistenceManagers**.

A **PersistenceManager** embodies a connection to a datastore and a cache of in-memory persistent objects. The **PersistenceManager** interface is the primary means by which the application interacts with the underlying datastore and in-memory persistent objects.

From a **PersistenceManager** the application can get one or more **Query** instances. A **Query** is how the application can find a persistent object by its field values.

In addition to a **Query** the application can get an **Extent** from a **PersistenceManager**. An **Extent** represents all the instances of a specified class (and optionally subclasses) stored in the datastore and can be used as input to a **Query** or iterated through in its own right.

A **Transaction** allows the application to control the transaction boundaries in the underlying datastore. There is a one-to-one correlation between a **Transaction** and a **PersistenceManager**, there can be only one ongoing transaction per **PersistenceManager** instance. Transactions must be explicitly started and either committed or aborted.

3.3 DEFINING A CLASS

The first step in using JDO is to create a persistence capable Java class. For the purposes of the examples in the book it is assumed that byte code enhancement is going being used, so the first step is simply to write a Java class. The following code snippet taken from “Author.java” shows the code for a class called Author, which represents the author of a book:

```
package com.corejdo.examples.chapter4.model;

public class Author {

    private String name;

    public Author (String name) {

        this.name = name;
    }

    protected Author () {} // Required by JDO

    public String getName () {

        return name;
    }

    public void setName (String name) {

        this.name = name;
    }
}
```

This class is relatively simply, with just one string field. JDO can of course support much more than this, see section 3.9 on the JDO Object Model for more details on what is supported.

Note that the fields of Author are declared as private. Unlike other approaches to object persistence, JDO does not require fields of classes to be declared public. In fact JDO places very few constraints on the definition of the Java classes themselves. The one thing that is required is a default no-args constructor.

No-args Constructor

JDO requires that a persistence capable class have a default no arguments (no-args) constructor. This constructor does not need to be declared public, it just needs to be accessible to the

class itself and potentially any subclasses. If there are no subclasses it can be declared as private. However if there are subclasses then it should be declared as protected or package so that it is accessible from a subclasses no-args constructor.

The no-args constructor is used by the JDO runtime to create empty instances of the class prior to retrieving its field values from the datastore.

To make the Author class persistence capable it needs to be compiled and passed to the byte code enhancement tool. But before this can be done an XML file needs to be created that contains the JDO metadata for the class.

JDO Metadata

For each persistence capable class JDO requires additional metadata, this metadata is specified in an XML file. The metadata is primarily used when making a Java class persistence capable (by byte code or source enhancement or code generation), however it is likely to also be used by a JDO implementation at runtime.

By convention these files have a “.jdo” suffix and there can be a metadata file per class, in which case the file is named after the class itself, or there can be one per package, in which case the file is named “package.jdo”. The metadata files should be accessible at runtime as resources via the class loader that loaded the persistence capable Java class.

For the Author class, a file called “Author.jdo” would be located in the same directory as the Author class file, this allows the byte code enhancement and JDO runtime to easily locate it:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="Author"/>
  </package>
</jdo>
```

Alternatively the metadata for this class could be located in a file called “package.jdo” which would be located in the “com/corejdo/examples/chapter/model” directory. This file could contain metadata on all the classes in the “model” package (and sub-packages), not just the Author class. [Appendix X](#) provides a detailed overview of the syntax for the JDO metadata, for this class however all that is needed is to specify package name and class name.

Metadata Filenames

The initial JDO 1.0 specification didn't actually mandate the location and name of the XML metadata files. To aid in portability the X.X.X maintenance revision to the specification changed this to what has just been described.

Prior to this revision the name of a metadata file that contained multiple classes was the package name itself with a ".jdo" suffix and it would have been located in the directory containing the package directory. In the Author example it would have been called "model.jdo" and would have been located in the "com/corejdo/examples/chapter" directory. Some JDO implementations may still use this naming convention.

Once the Author source and metadata files are ready they can be passed to the appropriate vendor's byte code enhancement tools. These will take care of implementing the persistence capable contract. As an example, if using the JDO reference implementation (RI), the Author class would be compiled, as normal, using "javac" and then the RI byte code enhancement tool would be invoked from the command line as follows:

```
javac Author.java
java com.sun.jdori.enhancer.Main Author.class Author.jdo
```

By default, the RI enhancement tool updates the Author byte code in-place and the original "Author.class" file is replaced with an enhanced one.

Mapping a Class to the Datastore

JDO does not define how to specify how the fields of a class should actually be mapped to the underlying datastore, i.e. if using a relational database what tables and column should be used. It does not define how the datastore schema should be created in the first place either. All this is datastore and JDO implementation specific. If required a vendor will likely provide additional tools to allow a developer to define a mapping between the persistence capable classes and the underlying datastore and to define the required datastore schema.

As an example, a JDO implementation on top of a relational database might use "vendor extensions" to the JDO metadata for a class to specify table and column names or define how field values should be stored. An implementation using an object database or a flat file might only require the Java classes and nothing more.

3.4 CONNECTING TO A DATASTORE

Once a persistence capable class has been defined the next step is to implement an application that actually uses it. Any JDO application needs to initially get a connection to the underlying datastore before it can do anything. This is done by configuring an instance of `PersistenceManagerFactory` and then using it to get a `PersistenceManager`.

`PersistenceManagerFactory` is just a Java interface; JDO does not really mandate how to create an actual `PersistenceManagerFactory` instance. It does however provide a helper class called `JDOHelper` that amongst other things provides a common bootstrap mechanism to create a `PersistenceManagerFactory` instance based on a specified set of properties:

```
static public PersistenceManagerFactory  
    getPersistenceManagerFactory(Properties props)
```

`JDOHelper.getPersistenceManagerFactory()`

This method takes a set of properties as input and constructs a concrete instance of a `PersistenceManagerFactory`. The one required property is, “`javax.jdo.PersistenceManagerFactoryClass`”, which is used to identify the name of the vendor’s `PersistenceManagerFactory` class. All other properties are simply passed to the vendor’s class and are vendor specific. These include things like connection URL, name and password.

As a minimum every vendor has to support bootstrapping via `JDOHelper`. Additionally a JDO vendor might provide their own constructors on their `PersistenceManagerFactory` class that can be used to directly construct a `PersistenceManagerFactory` instance. They may also provide constructors on their `PersistentManager` class to avoid needing to go through a `PersistenceManagerFactory` at all. From a portability standpoint these approaches are all vendor specific.

`PersistenceManagerFactory` also implements `Serializable`, the intention being to allow a previously configured `PersistenceManagerFactory` instance to be looked up via JNDI.

Once a `PersistenceManagerFactory` instance has been created the `getPersistenceManager()` method can be used to get a `PersistenceManager` instance:

```
public PersistenceManager getPersistenceManager()
```

When an application is finished with a `PersistenceManager` instance it should close it by calling the `close()` method:

```
public void close()
```

Connection Pooling

JDO leaves it up to the vendor as to whether their `PersistenceManagerFactory` implements connection pooling. If implemented the `close()` method on `PersistenceManager` may not physically close anything but instead return the `PersistenceManager` instance to the `PersistenceManagerFactory` to be re-used. For more details see [chapter X, section X.X](#) on `PersistenceManagerFactory`.

The following code snippet taken from “`MakeConnectionExample.java`” shows how to get a `PersistenceManager` instance from a `PersistenceManagerFactory` and then close it:

```
package com.corejdo.examples.chapter4;

import java.util.Properties;
import javax.jdo.*;

public class MakeConnectionExample {

    public static void main(String[] args) {

        Properties properties = new Properties();

        properties.put(
            "javax.jdo.PersistenceManagerFactoryClass", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionURL", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionUserName", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionPassword", "XXX");

        PersistenceManagerFactory pmf =
            JDOHelper.getPersistenceManagerFactory(properties);

        PersistenceManager pm = pmf.getPersistenceManager();

        /* Do something interesting */

        pm.close();
    }
}
```

Specifying Properties

As an alternative to hard coding the properties as in the above example, its generally better to specify them as system properties or read them from a file. The examples directory on the CD for this chapter contains examples on how these two alternative approaches would work. See:

“MakeConnectionFromSystemPropertiesExample.java” and

“MakeConnectionFromFileExample.java”

For a full list of the standard JDO property names that can be specified see **chapter X, section X.X** on PersistenceManagerFactory.

3.5 CREATING AN OBJECT

Once a PersistenceManager has been created it is possible to actually begin a transaction and do something interesting with the datastore. To begin a transaction, get the current Transaction instance from the PersistenceManager:

```
public Transaction currentTransaction()
```

And call the begin method on Transaction to start a datastore transaction:

```
public void begin()
```

Transactions

Typically any interaction with a datastore is done within the context of a transaction. A transaction simply marks the beginning and end of a unit of work, whereby all changes made will either happen or not happen.

A transaction is also mechanism a datastore uses to enforce concurrency control between multiple applications accessing the datastore at the same time. Not all datastores support concurrent access, but those that do will ensure that each ongoing transaction is isolated from the others. For more details on concurrency control and transactions see section 3.13.

Instances of persistence capable classes can now be created and made persistent. They are constructed as per normal Java and once constructed need to be passed to the makePersistent() method on PersistenceManager to notify JDO that the instances need to be persisted:

```
public void makePersistent(Object pc)
```

The instances won't actually be stored in the datastore until the transaction is committed, this is done using the `commit()` method on `Transaction`:

```
public void commit()
```

Alternatively a transaction can be rolled back, in this case the instances won't become persistent and won't be stored in the datastore, this can be done using the `rollback()` method on `Transaction`, instead of `commit()`:

```
public void rollback()
```

The following code snippet taken from "CreateExample.java" shows how to create an instance of the `Author` class and make it persistent in the datastore (assuming the `PersistenceManager` has already been created as per the previous example):

```
Transaction tx = pm.currentTransaction();

tx.begin();

Author author = new Author("Keiron McCammon");

pm.makePersistent(author);

tx.commit();
```

Persistence by Reachability

In Chapter 1 we introduced the concept of persistence by reachability and persistent instances. Not every instance that needs to be persistent has to be passed to `makePersistent()`. JDO defines that any instance of a persistence capable class that is reachable from a persistent object will automatically become persistent also.

This means that only the root of a graph of objects actually needs to be passed to `makePersistent()`, all other objects in the graph will implicitly become persistent also. This makes programming much easier.

Once successfully committed the fields of the new `Author` instance are stored in the datastore and the in-memory instance becomes what is termed "hollow". This means that its fields are cleared so that when the instance is used in the next transaction they will be retrieved again from the datastore.

Object Lifecycle

JDO defines a number of states for persistent objects, these states are used by the JDO runtime to manage the in-memory lifecycle of persistent objects. Whilst it is not necessary to really understand the JDO object lifecycle (its really of primary concern to the JDO vendors) an appreciation of what happens under the covers is always a good idea. Section X.X on Object Lifecycles for more details.

3.6 READING AN OBJECT

Once an instance has been made persistent and its fields stored in the datastore it can be retrieved again at some later point in time, either within the same application or by a different application.

There are three primary ways of finding an instance in the datastore with JDO, by navigation, via an extent or by a query.

3.6.1 Reading by Navigation

Retrieving an instance by navigation is simple. Extending the previous example, this code snippet taken from “ReadByNavigationExample.java” shows how the Author instance can be retrieved again in the next transaction simply by using it:

```
tx.begin();

String name = author.getName();

System.out.println("Author's name is '" + name + "'.");

tx.commit();
```

Underneath the covers the JDO runtime automatically retrieves the Author instance's fields from the datastore. The output would be:

```
Author's name is 'Keiron McCammon'.
```

Using navigation to retrieve instances from the datastore also applies to instances that are referenced from any persistent object that has been retrieved into memory. Essentially anytime a persistent object is touched the JDO runtime automatically retrieves the instance's fields from the datastore, if it has not already done so within the current transaction.

Transaction Boundaries

A common question arises from the previous example, why do the fields of the Author instance have to be fetched from the datastore again after it was initially created?

The answer has to do with transactions. After each transaction ends (either by a commit or rollback) all in-memory persistent objects become hollow. This is because after a transaction ends the representation of a persistent object in the datastore could be updated by another application, therefore making any in-memory values stale. For this reason JDO's default behavior is to clear the fields of each persistent object and re-retrieve them if used within the next transaction, thereby ensuring the in-memory instance remains consistent with the datastore.

JDO has some advanced options that change this default behavior and allow instances to be retained over transaction boundaries or even accessed outside of a transaction altogether. See chapter X, section X.X on Optional Features for more details.

3.6.2 Reading using an Extent

What happens if another application wants to access the new Author instance, obviously this application won't have a reference that it can navigate to. In this case the application can use an Extent to find all Authors.

Extents

An Extent represents a collection of all instances in the datastore of a particular persistence capable class and can consist of just instances of the class or instances of all subclasses also. An application can get an Extent for a class from a PersistenceManager and then use an Iterator to iterate through all instances or use the Extent as input to a query.

By default it is possible to get an Extent for any persistence capable class. If an Extent is not required then as a potential optimization, it is possible to specify explicitly that an extent is not required in the JDO metadata for the class. Depending on the underlying datastore this may or may not make any difference.

An application can get an Extent using the `getExtent()` method on PersistenceManager:

```
public Extent getExtent(Class pc, boolean subclasses)
```

The `subclasses` flag determines whether the returned Extent represents instances of the specified class only or includes instances of subclasses as well. If true then it will include instances of all subclasses as well.

The `iterator()` method on Extent can be used to get an Iterator that can be used to iterate through all the instances in the Extent:


```
public Iterator iterator()
```

The following code snippet taken from “ReadByExtentExample.java” shows how an Extent can be used to iterate through the instances of a particular class:

```
tx.begin();

Extent extent = pm.getExtent(Author.class, true);

Iterator iter = extent.iterator();

while (iter.hasNext()) {

    Author author = (Author) iter.next();

    String name = author.getName();

    System.out.println("Author's name is '" + name + "'.");
}

extent.closeAll();

tx.commit();
```

Its good practice to use the close() or closeAll() methods on Extent when an iterator has been used. The close() method closes an individual iterator whereas closeAll() closes all iterators retrieved from the Extent:

```
public void close(Iterator iterator)
public void closeAll()
```

Depending on the underlying datastore this may free resources that were allocated when the Iterator was created (a cursor for example).

3.6.3 Reading using a Query

An Extent is useful if an application wants to retrieve all instances of a class in no particular order. However if an application is looking for a particular instance that has certain field values, an Extent is not very useful. In this case a query is needed to find an instance based on the values of its fields.

JDO defines a query language called JDOQL (JDO Query Language). We will look at JDOQL in detail in Chapter X, however it can be thought of as essentially being a simplified

version of the Java syntax for multiple “if” statement. The PersistenceManager is used to create a Query instance and once created a Query can be executed to return a collection of the matching persistent objects. There are around nine different methods on PersistenceManager to create a Query, the most straightforward just requires a Java class and a filter:

```
public Query newQuery(Class cln, String filter)
```

The class specifies the persistence capable class that the query is for and the filter, which is just a string, specifies which instances of the class are to be returned.

Using an Extent

A Query can also be created using an Extent. A query created using a Java class directly is equivalent to a Query created using an Extent, where the Extent was created using subclasses as true. This means that a Query created from a Java class may include instances of the specified class or subclass (which is generally the desired behavior).

If an application specifically wants to restrict the query to instances of a given class only, then it can create an Extent with subclasses as false and use the Extent to create the Query. When executed the Query will only include instances of the class itself and will ignore subclasses.

The basic syntax for a filter is:

```
<field> <operator> <constant>
```

Where <field> is the name of the field as declared in the Java class, <operator> is one of the standard Java comparison operators (==, !=, <, >, and so on) and <constant> is the value to be compared against. Much more sophisticated queries than this can of course be defined, see [Chapter X](#) on Query for more details.

The following code snippet taken from “ReadByQueryExample.java” shows how to use a query to find a previously created Author by name:

```
tx.begin();

Query query =
    pm.newQuery(Author.class, "name == \"Keiron McCammon\"");

Collection result = (Collection) query.execute();

Author author = (Author) result.iterator().next();

query.close(result);

String name = author.getName();
```

```
System.out.println("Author's name is '" + name + "'.");  
  
tx.commit();
```

The output would be:

Author's name is 'Keiron McCammon'.

The `execute()` method on `Query` is used to execute the query and return the results, there are half a dozen ways of executing a query, the simplest is:

```
public Object execute()
```

The `execute()` methods all return `java.lang.Object` which needs to be cast to a `java.util.Collection` type for JDOQL queries. This collection is the set of instances that matched the specified filter and can be iterated through in the normal manner to retrieve the persistent objects themselves.

Query.execute()

The reason the `execute()` methods on `Query` return `java.lang.Object` rather than `java.util.Collection` directly is to allow for vendor extensions.

JDO mandates that JDOQL must at least be supported, however vendors can optionally support alternative query languages (maybe SQL, OQL or something proprietary). In this case the result of executing a query may not be a `java.util.Collection`, instead it would need to be cast to something specific to the query language being used.

It is good practice to use the `close()` method on `Query` to release the result returned from a call to `execute()`:

```
public Object close(Object result)
```

Depending on the underlying datastore this may free resources that were allocated during the execution of the query (a cursor for example).

3.7 UPDATING AN OBJECT

It is possible to modify a persistent object as per normal Java with no additional coding required. Assuming the class defines methods that can be used to set the fields then all the application needs to do is invoke these methods within a transaction. The JDO runtime auto-

automatically detects when a field of a persistent object has been modified and marks the field as “dirty” for that instance. At commit time the dirty fields of all the modified instances are written back to the datastore as part of the transaction.

The following code snippet taken from “UpdateExample.java” shows how to find a previously created Author instance and change its name. To validate the name has changed in the datastore it is printed in a new transaction:

```
tx.begin();

Query query =
    pm.newQuery(Author.class, "name == \"Keiron McCammon\"");

Collection result = (Collection) query.execute();

Author author = (Author) result.iterator().next();

query.close();

author.setName("Sameer Tyagi");

tx.commit();

tx.begin();

String name = author.getName();

System.out.println("Author's name is '" + name + "'.");

tx.commit();
```

The output would be:

```
Author's name is 'Sameer Tyagi'.
```

If it is decided that the changes should not be written to the datastore and they should be discarded, then the `rollback()` method on `Transaction` can be used.

The following code snippet taken from “UpdateExampleWithRollback.java” is the same as the previous one, except a rollback is used instead of a commit to undo the name change:

```
tx.begin();

Query query =
    pm.newQuery(Author.class, "name == \"Keiron McCammon\"");
```

```
Collection result = (Collection) query.execute();

Author author = (Author) result.iterator().next();

query.close();

author.setName("Sameer Tyagi");

tx.rollback();

tx.begin();

String name = author.getName();

System.out.println("Author's name is '" + name + "'.");

tx.commit();
```

The output would be:

Author's name is 'Keiron McCammon'.

3.8 DELETING AN OBJECT

The last basic operation is object deletion. Unlike Java most datastores do not perform automatic garbage collection. Indeed the semantics of what constitutes garbage when instances are made persistent changes. Just because a persistent object is no longer referenced by any other it does not mean it is garbage, typically it can always be retrieved at any time via Query or Extent.

JDO therefore provides a means to explicitly delete persistent objects. The `deletePersistent()` method on `PersistenceManager` can be used to remove persistent objects from the datastore:

```
public void deletePersistent(Object pc)
```

Of course the persistent object isn't actually deleted until the transaction is committed, rather the JDO runtime marks the instance as deleted and upon commit, requests that the datastore remove all the deleted instances. The following code snippet taken from "DeleteExample.java" shows how to delete an instance of Author:

```
tx.begin();

Query query =
    pm.newQuery(Author.class, "name == \"Keiron McCammon\"");

Collection result = (Collection) query.execute();

Author author = (Author) result.iterator().next();

query.close(result);

pm.deletePersistent(author);

tx.commit();
```

Trying to access the fields of a deleted instance within the same transaction will result in `JDOUserException` being thrown

After the transaction commits, the in-memory instance reverts to being a normal transient Java object, with its fields reset to their default values. If a rollback is called instead then the object deletion is undone and the in-memory instance reverts to being a “hollow” persistent object. The following code snippet taken from “DeleteWithRollbackExample.java” uses a rollback rather than a commit and then validates the persistent object still exists by printing its name:

```
pm.deletePersistent(author);

tx.rollback();

tx.begin();

String name = author.getName();

System.out.println("Author's name is '" + name + "'.");

tx.commit();
```

The output would be:

Author's name is 'Keiron McCammon'.

PersistenceManager.deletePersistent()

Unlike normal Java, where garbage collection automatically determines if an instance is still referenced or not and if not frees it. With persistent objects it is down to the application to ensure unwanted persistent objects are deleted and that an instance being deleted is no longer referenced by another instance. Depending on the datastore, it may be possible to define constraints to guard against deleting instances that are still referenced, but this is implementation specific. If an instance does reference a previously deleted instance then `JDOUserException` is thrown when the deleted instance is later accessed (since it will no longer be found in the datastore).

Unlike when making instances persistent, deletion has no equivalent of persistence by reachability. Deleting an instance deletes only the specified instance, it does not delete any referenced instances also.

It is possible for an application to implement a cascading delete behavior by using the `JDO InstanceCallbacks` interface and implementing the `jdoPreDelete()` method to explicitly delete referenced instances. See chapter X, section X.X on `InstanceCallbacks` for more details.

3.9 JDO OBJECT MODEL

So far the examples have used a very simple Java class to demonstrate basic concepts. JDO can of course be used with much more sophisticated object models than this. In addition to supporting fields of primitive Java types and system classes like `String`, JDO can handle object references, Java collections and inheritance of both classes and interfaces.

Optional Features

JDO defines both mandatory and optional features. A JDO implementation can be said to be compliant if it supports all the mandatory features defined by the JDO specification. A vendor can choose to support one or more optional features, if so then their implementation must adhere to what is defined for the optional feature by the specification to be called JDO compliant.

From a portability standpoint, an application should not rely on support of an optional feature. But should a particular feature be required then at least the application will still be portable across vendors that support the feature or set of features.

See chapter X, section X.X on optional features for more details.

3.9.1 Basic Types

A persistence capable class can have fields whose type's can be any primitive Java type, primitive wrapper type or other supported system immutable and mutable classes. Table 3.1 provides a summary of these basic types:

Table 3.1 Basic Types

Primitive Types	Wrapper Classes	Supported System Interfaces	Supported System Classes
boolean	java.lang.Boolean [¥]	java.util.Collection [§]	java.lang.String [¥]
byte	java.lang.Byte [¥]	java.util.List ^{*§}	java.math.BigDecimal [¥]
char	java.lang.Character [¥]	java.util.Map ^{*§}	java.math.BigInteger [¥]
double	java.lang.Double [¥]	java.util.Set [§]	java.util.Date
int	java.lang.Integer [¥]	javax.jdo.PersistenceCapable	java.util.Locale [¥]
float	java.lang.Float [¥]		java.util.ArrayList ^{*§}
long	java.lang.Long [¥]		java.util.HashMap ^{*§}
short	java.lang.Short [¥]		java.util.HashSet [§]
			java.util.Hashtable ^{*§}
			java.util.LinkedList ^{*§}
			java.util.Map ^{*§}
			java.util.TreeMap ^{*§}
			java.util.TreeSet ^{*§}
			java.util.Vector ^{*§}

*Support is an optional feature.

§See section X.X on Collections for more details.

¥Immutable classes.

If a persistence capable class has fields of any of the above types then by default they will be persisted in the datastore, likewise for any references to a persistence capable class. Fields of any other types will not be persisted in the datastore by default and need explicit metadata to indicate they should be persisted. The types of these fields might be a reference to Java Interface, java.lang.Object, non-supported system class/interface or non-persistence capable class.

If a persistence capable class references one or more instances of a wrapper classe or supported system classes, then since these instances cannot themselves be persistence capable they are treated as embedded instances by the JDO runtime.

3.9.1.1 Embedded Objects

Embedded instances only exist as part of a referencing persistent object, they typically have no identity in a datastore and usually can't be fetched independently of the referencing persistent object (although both of these are implementation specific).

An instance will be treated as embedded if:

- It is an instance of a non-persistent capable user defined class
- It is an instance of a supported system class.
- It is an array.
- It is an instance of a persistence capable class assigned to a field that has been explicitly marked as embedded in the JDO metadata for the class.

An application can rely on the following behavior when dealing with embedded instances:

- **Deletion** Embedded instances will be implicitly deleted when their referencing persistent object is deleted, the application is not responsible for deleting embedded instances. Although, depending on the underlying datastore the embedded instances may actually be garbage collected at a later time rather than being actually deleted. This is true even for embedded instances of persistence capable classes.
- **Extent and query visibility** Embedded instances will not be part of an Extent and can't be retrieved independently of their referencing persistent object via a Query. For example, embedded instances of a persistence capable class will not be part of the classes Extent and won't be visible in a Query on the class.
- **Query filters** Fields of embedded instances of persistence capable classes will be visible in a Query on the referencing class. This means a Query filter can use the dot notation to reference a field of an embedded instance.
- **Updates** Changes to embedded instances of supported systems classes or persistence capable classes are automatically tracked and the referencing persistent object is notified of the modification. However, an application needs to explicitly notify the referencing persistent object when changes are made to instances of non-persistence capable classes or unsupported system classes.

When a persistent object is stored in the datastore essentially a copy is made of all embedded instances, which are then stored along with the persistent object in the datastore. This means that if multiple persistent objects reference the same embedded instance, when written to the datastore each would make its own copy. When retrieved again, each persistent object will reference its own embedded instance. However, an application cannot depend on this treatment. Specifically, if embedded instances are shared when created (or updated), they might continue to be shared thereafter.

Automatic Tracking

When using the supported mutable system classes (see Table 3.1) changes are automatically tracked because the instances are replaced by “tracked” instances by the JDO runtime. These “tracked” instances are simply instances of classes provided by the JDO runtime that extend the system classes and add additional behavior to manage when changes are made and notify the referencing persistent object automatically.

When an embedded instance belongs to a persistence capable class the instance is already able to detect when changes are made and can therefore notify the referencing persistent object automatically.

The JDO runtime has no way of automatically tracking changes to any other class, therefore in this situation the application must explicitly notify the referencing persistent object when a change is made using the `makeDirty()` method.

3.9.2 References

A persistence capable class can have fields that reference other persistence capable classes.

The following code snippet taken from “BookWithAuthor.java” shows a class that has an “author” field that is a reference to an instance of Author:

```
package com.corejdo.examples.chapter4.model;

public class BookWithAuthor {

    private String name;
    private Author author;

    public BookWithAuthor(String name, Author author) {

        this.name = name;
        this.author = author;
    }

    protected BookWithAuthor() {} // Required by JDO

    public String getName() {

        return name;
    }

    public void setName(String name) {
```

```
        this.name = name;
    }

    public Author getAuthor() {

        return author;
    }

    public void setAuthor(Author author) {

        this.author = author;
    }
}
```

The JDO metadata for this class in “BookWithAuthor.jdo” is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="BookWithAuthor"/>
  </package>
</jdo>
```

The following code snippet taken from “CreateWithReachabilityExample.java” shows persistence by reachability at work. An Author instance is created and used to create a new BookWithAuthor instance. Only the new BookWithAuthor instance is explicitly made persistent (using makePersistent()), the Author is automatically make persistent to because it is reachable from the BookWithAuthor instance:

```
tx.begin();

Author author = new Author("Keiron McCammon");

BookWithAuthor book =
    new BookWithAuthor("Core Java Data Objects", author);

pm.makePersistent(book);

tx.commit();
```

As well as supporting references to persistence capable classes JDO also supports references to Java interfaces and even `java.lang.Object`. In these cases additional metadata is required to denote that the field should be persistent, since fields of these types are not persistent by default. Any referenced instances should still belong to a persistence capable class (support for references to non-persistence capable classes is JDO optional feature).

The following code snippet taken from “BookWithAuthorAsObject.java” shows a class that has an “author” field which is of type `java.lang.Object` instead of `Author`:

```
package com.corejdo.examples.chapter4.model;

public class BookWithAuthorAsObject {

    private String name;
    private Object author;

    /* Rest of code not shown */
}
```

The major difference compared to the previous example is the metadata in “BookWithAuthorAsObject.jdo”:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="BookWithAuthorAsObject">
      <field
        name="author"
        persistence-modifier="persistent"/>
    </class>
  </package>
</jdo>
```

Because the field’s type is `java.lang.Object` it is not persistent by default and has to be explicitly denoted as persistent in the class’s metadata. The same would be true if referencing an interface.

3.9.2.1 References to Non-Persistence Capable Classes

A JDO implementation may optional support references to instances that do not belong to a persistence capable class. If not supported a Java class cast exception would be thrown at

the time of assignment (an application intended to run against multiple JDO implementations should not rely on support for persisting non-persistence capable classes).

If references to non-persistence capable classes are supported then these instances are treated as embedded, in that they only exist in the datastore as part of the referencing persistent object (see section X.X on embedded objects for more details).

Because instances of non-persistence capable classes are treated as embedded objects it is the responsibility of the application to inform the referencing persistent object of any modification, if this isn't done then any changes to the embedded instance will not be written to the datastore on commit. The easiest way to notify a referencing persistent object of a change is to use the `makeDirty()` method on `JDOHelper`:

```
static void makeDirty(Object pc, String field)
```

The field argument is the name of the class's field that references the non-persistence capable instance.

JDOHelper

JDOHelper was previously introduced as a way to bootstrap a JDO application and get an instance of a `PersistenceManagerFactory`. As well as this `JDOHelper` has a number of additional utility methods that act as short cuts to the various JDO APIs.

The following code snippet taken from "Address.java" shows a non-persistence capable class called `Address` that will be used to store the address of an `Author`:

```
package com.corejdo.examples.chapter4.model;

import java.io.Serializable;

public class Address implements Serializable {

    private String street;
    private String zip;
    private String state;

    public Address(String street, String zip, String state) {

        this.street = street;
        this.zip    = zip;
        this.state  = state;
    }

    /* Additional getters and setters not shown */
```

```
public void setZip ( String zip ) {  
    this.zip = zip;  
}  
}
```

Managing Non-persistence Capable Classes

Exactly how a JDO implementation manages the fields of non-persistence capable classes is undefined by JDO. Some implementations may require that a class implement Serializable or follow the JavaBean pattern or its fields be declared public.

The following code snippet taken from “AuthorWithAddress.java” shows how to set the zip code of the address. The method first calls `makeDirty()` to inform the JDO runtime that the address is going to be modified:

```
package com.corejdo.examples.chapter4.model;  
  
import javax.jdo.*;  
  
public class AuthorWithAddress {  
    private String name;  
    private Address address;  
  
    public AuthorWithAddress(String name, Address address) {  
        this.name = name;  
        this.address = address;  
    }  
  
    protected AuthorWithAddress() {} // Required by JDO  
  
    public void setZip(String zip) {  
        JDOHelper.makeDirty(this, "address");  
        address.setZip(zip);  
    }  
}
```

JDOHelper.makeDirty()

It is best practice to call `makeDirty()` before making any modifications to an embedded instance. Depending on the underlying datastore the call to `makeDirty()` may result in a concurrency conflict indicating that changes aren't allowed at that time.

The JDO metadata for this class, in “AuthorWithAddress.jdo”, is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="AuthorWithAddress">
      <field
        name="address"
        persistence-modifier="persistent"/>
    </class>
  </package>
</jdo>
```

Because the address field is a reference to a non-persistence capable class it is necessary to specifically specify that the field should be persistent.

Tracking changes to Non-Persistence Capable Objects

When using non-persistence capable objects it is best to only modify these objects through methods on the referencing persistent object. This way the persistent object can call `makeDirty()` on itself before making any change.

If a non-persistence capable object is passed by reference and modified elsewhere it becomes hard for the application to ensure the referencing persistent object is notified of the changes, in which case any changes will be ignored.

3.9.2.2 Collection Classes

A persistence capable class can have fields that reference instances of the standard Java collection classes. Instances of these collection classes are treated as embedded objects, in that they only exist in the datastore as part of the referencing persistent object (see section 3.9.1.1 on embedded objects for more details). As a minimum JDO mandates support for `java.util.HashSet`, support for `ArrayList`, `HashMap`, `Hashtable`, `LinkedList`, `TreeMap`, `TreeSet` and `Vector` are JDO optional features. However, just about all implementations support most of these collection classes.

Optional Features

To determine if an implementation supports the optional collection classes use the `supportedOptions()` method on `PersistenceManagerFactory`. See [chapter X, section X on PersistenceManagerFactory](#) for more details.

The following code snippet taken from “AuthorWithBooks.java” uses a `java.util.HashSet` to hold the books that an author has written:

```
package com.corejdo.examples.chapter4.model;

import java.util.*;

public class AuthorWithBooks {

    private String name;
    private Set    books = new HashSet();

    public AuthorWithBooks(String name) {

        this.name = name;
    }

    protected AuthorWithBooks() {} // Required by JDO

    public String getName() {

        return name;
    }

    public void setName(String name) {

        this.name = name;
    }

    public void addBook(Book book) {

        books.add(book);
    }

    public Iterator getBooks() {
```



```
        return books.iterator();
    }
}
```

The JDO metadata for this class in “AuthorWithBooks.jdo”, is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="AuthorWithBooks">
      <field name="books">
        <collection
          element-type=
            "com.corejdo.examples.chapter4.model.Book"/>
      </field>
    </class>
  </package>
</jdo>
```

Because this class uses a Java collection additional metadata can be used to declare the type of the objects that will be stored in the collection. Since Java collections are un-typed it is impossible for the JDO runtime to determine this information from the Java class itself.

Additional Meta-data

The additional metadata shown for the “books” field is optional, it does not need to be specified. If not specified the JDO runtime will assume that the collection may contain a reference to any object, as per normal Java.

Even though it is optional it is good practice to define the element type of a collection as it potentially allows the JDO runtime to optimize how it handles the field both in-memory as well as in the datastore.

Unlike user defined, non-persistence capable classes, JDO mandates that the supported collections automatically detect modifications and notify the referencing persistent object. The way a JDO runtime does this is to replace instances of system defined collection class with its own, equivalent classes that perform this detection. For a new object, this replacement will happen during the call to `makePersistent()`, for existing persistent objects when retrieved from the datastore. All this is completely transparent to the application, it just means that the application should not rely on a collection being a particular concrete Java class.

3.9.3. Arrays

Support for a persistence capable class with fields that use Java arrays is optional in JDO. If supported, arrays are similar in nature to non-persistence capable objects except that a JDO runtime may optionally automatically detect changes and notify the referencing persistent object of the modification. For portability the application should assume responsibility for notifying the referencing persistent object when an array is modified.

As per non-persistence capable classes it is best practice to only modify arrays through methods on the referencing persistent object. This way the referencing persistent object can call `makeDirty()` on itself before making any change. If an array is passed by reference and modified elsewhere it becomes hard for the application to ensure the owning persistent object is notified of the changes, in which case any changes will be ignored.

The following code snippet taken from “`AuthorWithBooksAsArray.java`” shows a class that uses an array to store the books an author has written:

```
package com.corejdo.examples.chapter4.model;

import java.util.*;

public class AuthorWithBooksAsArray {

    private String name;
    private Book[] books = new Book[1];

    public AuthorWithBooksAsArray(String name) {

        this.name = name;
    }

    protected AuthorWithBooksAsArray() {} // Required by JDO

    public String getName() {

        return name;
    }

    public void setName(String name) {

        this.name = name;
    }

    public void addBook(Book book, int i) {
```

```
        JDOHelper.makeDirty(this, "books");

        books[i] = book;
    }

    public Book getBook(int i) {

        return books[i];
    }
}
```

The JDO metadata for this class in “AuthorWithBooksAsArray.jdo”, is

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class name="AuthorWithBooksAsArray"/>
  </package>
</jdo>
```

Because the field is an array of a persistence capable class no additional metadata is required, the field will be persistent by default.

3.9.4 Inheritance

JDO supports inheritance of both interfaces and classes. Since Java interfaces don’t define any fields there is nothing that the JDO runtime needs to worry about so Java interfaces can be used with no real consideration as far as JDO is concerned.

When using class inheritance (abstract or otherwise) JDO needs to be aware of the implementation hierarchy, since each class can define its own fields. In the usual case where all classes in the inheritance hierarchy are themselves persistence capable its straightforward. The only additional requirement is to denote the persistence capable superclass in the JDO metadata.

The following code snippet taken from “RevisedBook.java” shows a class that extends the Book class:

```
package com.corejdo.examples.chapter4.model;

public class RevisedBook extends Book {
```

```
private Book original;

public RevisedBook(String name, Book original) {
    super(name);
    this.original = original;
}

protected RevisedBook() {} // Required by JDO

public Book getOriginal() {
    return original;
}

public void setOriginal(Book original) {
    this.original = original;
}
}
```

The JDO metadata for this class in 'RevisedBook.jdo', is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE jdo SYSTEM "http://java.sun.com/dtd/jdo_1_0.dtd">
<jdo>
  <package name="com.corejdo.examples.chapter4.model">
    <class
      name="RevisedBook"
      persistence-capable-superclass=
        "com.corejdo.examples.chapter4.model.Book"/>
    </package>
  </jdo>
```

The additional metadata denotes the persistence capable superclass of the class. The fully qualified name of the superclass must be used even if the superclass is in the same package as the class.

It is possible for a persistence capable class to extend a non-persistence capable class. In this case, any fields defined by the non-persistence capable class are ignored by JDO and are not persisted in the datastore. It is also possible to have a persistence capable class extend a non-persistence capable class which itself extends a persistence capable class. In all

cases any fields of a non-persistence capable class are ignored by JDO, it is the responsibility of the application to manage these fields.

Non-Persistence Capable Superclasses

It is best to make all classes in an inheritance hierarchy persistence capable. This avoids added complications.

If this isn't possible and non-persistence capable classes have to be used then a persistence capable subclass should implement the InstanceCallbacks Interface and use the `jdoPostLoad()`, `jdoPreStore()` and `jdoPreClear()` methods to manage the fields of the non-persistence capable classes explicitly.

3.9.5 Modifiers

JDO supports all Java class and field modifiers: `private`; `public`; `protected`; `static`; `transient`; `abstract`; `final`; `synchronized` and `volatile`. The following caveats apply:

- A field declared as `transient` will not be persisted in the datastore by default. This default behavior can be overridden by explicitly declaring the field as persistent in the class's metadata.
- A field declared as `final` cannot be persisted in the datastore, its value can only be set during object construction.
- A field declared as `static` cannot be persisted in the datastore, it only has meaning at the class level within the JVM.

3.9.6 What isn't Supported

JDO supports most of what can be defined in Java. The major exception is that JDO can't make the Java system classes themselves persistence capable, rather it mandates support for specific system classes (as already outlined) as fields of a persistence capable class. Also classes that depend on inaccessible or remote state, such as those that use JNI or those that extend system defined classes, cannot be made persistence capable and cannot be used as fields of persistence capable classes either.

3.10 EXCEPTION HANDLING

All the examples so far have ignored the possibility of exceptions. JDO defines a set of exceptions that can be thrown by the JDO runtime. All JDO exceptions are declared as runtime exceptions since they can be thrown at anytime not just when calling JDO methods. As an example, navigating from one instance to another may result in an exception if there is a problem with accessing the datastore.

JDO classifies exceptions broadly into fatal or non-fatal exceptions, non-fatal exceptions indicate an operation failed but can be retried, whereas fatal exceptions indicate that the only recourse is to start again. Beyond this there are user exceptions (fatal or non-fatal), which indicate user error; datastore exceptions (fatal or non-fatal), which indicate a datastore error; an internal exception (fatal), which indicates a problem with the JDO implementation and an unsupported feature exception (non-fatal), which indicates the JDO implementation doesn't support a particular feature.

The following code snippet taken from "CreateWithExceptionsExample.java" shows how to catch any JDO exception using the JDOException base class:

```
package com.corejdo.examples.chapter4;

import java.util.Properties;
import javax.jdo.*;

public class CreateWithExceptionsExample {

    public static void main(String[] args) {

        Properties properties = new Properties();

        properties.put(
            "javax.jdo.PersistenceManagerFactoryClass", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionURL", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionUserName", "XXX");
        properties.put(
            "javax.jdo.option.ConnectionPassword", "XXX");

        PersistenceManagerFactory pmf =
            JDOHelper.getPersistenceManagerFactory(properties);
        PersistenceManager pm = null;

        try {
```

```
        pm = pmf.getPersistenceManager();

        Transaction tx = pm.currentTransaction();

        tx.begin();

        Author author = new Author("Keiron McCammon");

        pm.makePersistent(author);

        tx.commit();

        pm.close();
    }

    catch (JDOException e) {

        // Handle the exception
    }

    finally {

        if (pm != null && !pm.isClosed()) {

            if (pm.currentTransaction().isActive()) {

                pm.currentTransaction().rollback();
            }

            pm.close();
        }
    }
}
```

In this simple example it doesn't really matter if the transaction is rolled back or if the `PersistenceManager` is closed since it exits immediately thereafter anyway. However, in a normal application it is good practice to use a try/catch block and finally block to ensure the current transaction is rolled back and, if appropriate, the `PersistenceManager` instance is closed.

3.11 OBJECT IDENTITY

One of the key concepts defined in JDO is that of object identity, though in most cases a developer is not even aware it exists. Every persistent object has a JDO object identity. This identity associates the in-memory Java object with its representation in the underlying database. JDO ensures that there is only one in-memory representation of a given persistent object for a given PersistenceManager. This is known as “uniquing”. Uniquing ensures that no matter how many times a persistent object is found it will only have one in-memory representation, all references to the same persistent object within the scope of the same PersistenceManager instance will reference the same in-memory object.

The following code snippet taken from “UniquingExample.java” shows uniquing at work. It creates a new Author, begins a new transaction and finds the Author again using a query, the two references are then compared to validate they both refer to the same in-memory object.

```
tx.begin();

Author author1 = new Author("Keiron McCammon");

pm.makePersistent(author1);

tx.commit();

tx.begin();

Query query =
    pm.newQuery(Author.class, "name == \"Keiron McCammon\"");

Collection result = (Collection) query.execute();

Author author2 = (Author) result.iterator().next();

tx.commit();

if (author1 == author2)
    System.out.println("There is only one object in memory");
```

The output would be:

There is only one object in memory

However, since it is possible to create multiple `PersistenceManager` instances within a JVM, it is possible that a persistent object may have multiple in-memory representations at any given time, one per `PersistenceManager`. Each would have the same JDO object identity, but would be a different in-memory Java object. To determine if two in-memory objects represent the same persistent object their JDO object identities need to be compared. The JDO-Helper class provides a method to get the JDO object identity of an object:

```
static Object getObjectId(Object pc)
```

The returned object can be compared with another using the `equals()` method to determine if two in-memory objects represent the same persistent object in the datastore.

The following code snippet taken from “`ObjectIdentityExample.java`” creates an `Author` using one `PersistenceManager` and then using a different `PersistenceManager` finds the `Author` again. The two references are compared to validate they refer to different in-memory objects. The JDO identities are then compared to validate they do however represent the same persistent object:

```
tx1.begin();

Author author1 = new Author("Keiron McCammon");

pm1.makePersistent(author1);

tx1.commit();

PersistenceManager pm2 = pmf.getPersistenceManager();

Transaction tx2 = pm2.currentTransaction();

tx2.begin();

Query query =
    pm2.newQuery(Author.class, "name==\"Keiron McCammon\"");

Collection result = (Collection) query.execute();

Author author2 = (Author) result.iterator().next();

tx2.commit();

if (author1 != author2)
    System.out.println(
        "There are multiple objects in memory");
```

```
Object emp1Id = JDOHelper.getObjectId(author1);
Object emp2Id = JDOHelper.getObjectId(author2);

if (emp1Id.equals(emp2Id))
    System.out.println("But they represent the same Author");
```

The output would be:

```
There are multiple objects in memory
But they represent the same Author
```

JDO actually defines three types of object identity for persistent objects: datastore identity; application identity and non-durable identity. JDO mandates that either datastore or application identity must be supported (support for both is optional) and support for non-durable identity is optional. Most JDO vendors support datastore identity as their default and some also support application identity (notably those that are designed to run on top of a relational database).

3.11.1 Datastore Identity

Datastore identity is the simplest form of identity. The JDO object identity of a persistent object is assigned and managed by the JDO runtime, typically in conjunction with the underlying datastore. The identity is orthogonal to any of the field values of a persistent object.

There is nothing the application itself needs to do or provide when using datastore identity other than the Java classes themselves.

When to use Datastore Identity

Unless there is a specific need otherwise, datastore identity should always be used. It's the simplest and most portable form of JDO object identity, being widely supported by most vendors.

3.11.2 Application Identity

Application identity is more akin to the concept of a primary key. The JDO object identity of a persistent object is determined by the values of certain of its fields.

If using application identity the application needs to specify which fields are part of the primary key in the class's metadata and provide a class that can be used to represent the JDO object identity.

When to use Application Identity

Application identity is particularly useful if using JDO on top of a pre-defined datastore schema where primary keys have already been defined.

3.11.3 Non-durable Identity

Non-durable identity is essentially no identity. The JDO object identity of an object is only valid within the context of a given transaction, if the same object is accessed in a different transaction it may have a different JDO identity.

Non-durable identity should be used only when identity has no relevance, i.e. there is no need to maintain references between objects. Essentially it only makes sense to query for objects with non-durable identity.

When to use Non-durable Identity

There are few, if any, vendors that actually support non-durable identity. Its use is limited to a small number of use cases and as such most developers can safely ignore non-durable identity.

3.12 OBJECT LIFECYCLES

JDO defines a number of different states that an in-memory persistent object can be in. Principally these states are used by the JDO runtime to determine if the fields of an instance need to be retrieved from the datastore and if modified need to be written back. Together these states represent an in-memory persistent object's lifecycle with respect to JDO.

Chapter X describes an object's lifecycle states in greater detail, however in general only following basic states need to be understood:

- Transient
- Persistent
- Hollow

The following figure shows a simplified state diagram for these basic lifecycle states:

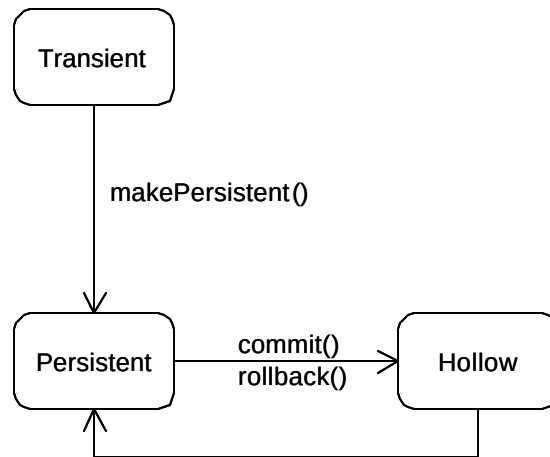


Figure 3.3 Simplified Lifecycle State Diagram

Transient denotes a normal, non-persistent Java object. When an instance of a persistence capable class is created it starts life as being transient. By default, JDO does not do anything to manage transient instances.

When made persistent, either explicitly via `makePersistent()` or because an instance is reachable from another persistent object, an instance transitions to being persistent (it actually transitions to a persistent-new state). There are actually five different states used to represent persistence, these allow the JDO runtime to differentiate between whether an instance is a new (persistent-new), has been modified (persistent-dirty) or deleted (persistent-deleted) and so on. All that really matters is that the instance is persistent, it has a JDO object identity and its fields have been retrieved from the datastore within the context of current transaction.

A hollow instance is also a persistent object (it has a JDO identity), however its fields have not yet been retrieved from the datastore, hence the term “hollow”. The first time a hollow instance is accessed its fields are retrieved from the datastore within the context of the current transaction and the instance transitions to being persistent (it actually transitions to persistent-clean to indicate the instance hasn’t yet been modified). Likewise, by default, after a transaction ends all instances that are persistent transition to being hollow.

There are several optional features that provide support for transient instances being managed as transactional objects (changes to their fields being undone on rollback) and persistent objects being accessed outside of transactions, see chapter X on object lifecycles for more details.

3.13 CONCURRENCY CONTROL

All the examples so far have been single user applications (only one transaction was accessing the datastore at a time). It is of course possible for multiple applications to be running concurrently. The JDO specification does not make any specific statements about concurrency control as it's a feature of the underlying datastore rather than of JDO.

JDO does define the concept of a “datastore transaction” and says that access to the datastore should be done within the context of a transaction and that these transactions should exhibit ACID properties. This allows the JDO implementation to use the underlying datastore's own concurrency control mechanisms, which maybe based on a pessimistic or optimistic locking scheme or some other approach. Because of this JDO does not expose APIs to explicitly lock persistent objects. If locks are required by the underlying datastore to enforce concurrency control then it is the responsibility of the JDO runtime to manage them implicitly. Also it is not possible to know when or how concurrency conflicts will be detected or resolved, all that can be guaranteed is that a transaction will be ACID in nature.

ACID Transactions

ACID stands for Atomic, Consistent, Isolated and Durable. It defines standard properties for a transaction:

Atomic – upon commit, either all changes will be in the datastore or none of them will be. This means that should something fail, arbitrary changes aren't left behind in the datastore.

Consistent – prior to and after commit, all data will be consistent as far as the datastore is concerned. This means that all data accessed during a transaction will initially be consistent and that after commit all changes will leave the data consistent also. Consistent means that the data does not violate any datastore constraints or invariants.

Isolated – prior to commit, any changes made will not be visible to other transactions and vice versa. This means that one transaction will not be able to see changes made by another transaction, it will only see committed changes.

Durable – after commit, any changes are guaranteed to persist even in the event of failure. This means that when a transaction ends any changes will be in the datastore.

JDO provides optional support for optimistic transactions. Optimistic transactions are useful for applications that have long running transactions where it is undesirable to hold datastore resources (such as locks) for an extended period of time. With optimistic transactions all concurrency control is deferred until the end of the transaction. Transactions are covered in detail in [Chapter X, please refer to section X.X](#) on Optimistic Transactions for more details.

3.4 SUMMARY

This chapter has provided an comprehensive introduction to using JDO, we outlined the basic concept of transparent persistence and explained what being persistence capable means for a class is and how byte code enhancement is a common approach to making a Java class persistence capable. It has shown how to create, read, update and delete persistent objects through simple examples and provided more sophisticated examples to show how JDO supports the full Java object model. It provided a brief overview of how to handle JDO exceptions and highlighted the advanced concepts of object identity, object lifecycles and concurrency control.

It should now be possible to develop and run an application using JDO. In the next chapter we will **look at X.**